

IBM OMPI Pitch

IBM Open Memory Protocol Initiative (OMPI) Pitch

Version 1, Sep 3, 2027

Why does IBM Care

- IBM's customers span the gamut of all regulated industries, and therefore IBM cares a great deal about data protection, ownership, provenance, and auditability
- IBM's customers are unwilling/unable to bring their data to existing hosted memory services, and therefore need the ability to use owned memory services
- As a company, IBM is walking the same path as our customers to modernize our use of AI. We are under the same regulatory constraints and pressures to transform our internal data into value with our employees' agents

Proposed Protocol Surface: Agentic memory

Interoperability between memory systems (import/export)

User Value

Users of memory systems can copy (and ideally move w/ deletion) their memories from one system to another, avoiding vendor lock-in

Interface Elements

- **Memory Body:** Raw text, no constraints
- **Memory Identifier:** Unique within the system it originated from
- **Lifecycle:** Information about creation including:
 - Author: Identifier for the human author involved in creation
 - AI usage / Provenance: What (if any) AI was used in creation and how
 - This could correlate to the EU AI Act
 - Timestamps:
 - Creation timestamp: ISO 8601 real-world timestamp when initially created
 - Invalidation timestamp: ISO 8601 timestamp when memory became / becomes invalid

- PROPOSAL => Memories are immutable, so no update timestamp is needed. Logical mutation is accomplished by override and invalidation. This allows “time travel” in the memory system versus data loss with in-place mutation.
 - Versioning: Versioning semantics left as implementation detail, but must have standard comparator implementation (eg numeric sequence number, semantic versioning, etc)
 - Source Material: (optional) linkage or raw source material to trace the original information used to create the memory
- **Semantic Tags:** List of arbitrary tags with semantic meaning (eg ontology, enum)
 - The tag values themselves should *not* be part of the standard, only what the schema of a tag set looks like
 - *Open Question:* Should tags be free-form text or more structured (eg well-defined enum for the system, or key/val ontology?)
 - As an example of implementation, JSON-LD <https://www.w3.org/TR/json-ld/> could be adopted to represent the meanings in the memory, where "@context" specifies the vocabulary (i.e. the meaning of the tags)
- **Scope Tags:** List of arbitrary tags defining memory scope
 - The tags themselves should not be defined, though this could certainly be discussed further with a more well-defined permission system
 - These tags will be linked to ACL policies

Interface Implications

- Schema for export/import well-defined
- Discontinuity between semantic and scope tags may necessitate a mapping/migration step during export/import
- ACLs bound to scope tags would need explicit mapping
- Memories have Write/Invalidate/Delete, but no Update (immutable)
- There is no guarantee of runtime interoperability between disjoint memory systems. In other words, a memory is valid in exactly one memory system at a time and when imported into another, it becomes a new memory in that system that is not valid in the old system.

Interoperability between memory systems and agents (runtime)

User Value

- Users of memory systems can attach their memories to multiple agents at the same time

- Memory system providers can provide agent-agnostic memory services via API contract
- Agent harness providers can consume memories from diverse memory providers without building them in-house

Interface Elements

- **Memory Attachment Formats:**
 - Write:
 - Remember: proactive tool-call
 - Observe: conversation hook (read-only)
 - Read:
 - Recall: proactive tool-call
 - Decorate: conversation hook (mutating)
- **Agent Attachment Surfaces:** Agents today support these formats in asymmetric ways
 - Tool calls: MCP
 - Plugins: Unstandardized
- **Remember Interface (MCP or file write):**
 - Memory body required
 - Semantic tags optional
 - Scope tags added systematically (no agent-defined scope tags)
- **Recall Interface (MCP or file read):**
 - Query seed
 - Timestamp optional (default to “now”) for temporal resolution
 - Semantic tags optional
 - Scope tags optional
 - System MUST validate requested scopes against ACLs
- **Observe Interface (hook):**
 - Conversation state in well-defined schema (in and out)
- **Decorate Interface (hook)**
 - Conversation state in a well-defined schema (in and out)

=> PROPOSAL: Hooks added to MCP as either a subset of tools with well-defined interfaces or as a standalone entity type